

Kapitel 7

Funktionen

7.1 Motivation

Je umfangreicher und komplexer ein Problem ist, desto schwieriger ist es zu lösen. Deshalb versucht man, Programmieraufgaben in verschiedene Teile zu zerlegen, die man ihrerseits wieder in Teilprobleme zerlegt, usw. Dabei achtet man besonders darauf, dass zwischen den Teilproblemen möglichst wenig Querverbindungen bestehen. Dieses Vorgehen hat drei Vorteile:

- Jedes Teilproblem ist viel einfacher zu lösen als das Gesamtproblem.
- Die Teilprobleme können von verschiedenen Programmierern gelöst werden.
- Wenn man „Glück“ hat, besitzt man schon Lösungen für einzelne Teilprobleme.

In der Terminologie des Programmierens entspricht eine solche Zerlegung eines Problems der Zerlegung eines Programms in Unterprogramme. C bezeichnet diese Unterprogramme grundsätzlich als *Funktionen*.

Wir haben bereits Funktionen kennengelernt und verwendet, nämlich die Bibliotheksfunktionen `printf`, `scanf`, `isspace`, usw. Auch das Hauptprogramm `main` ist in der Terminologie von C eine Funktion, die vom Betriebssystem beim Start des Programms aufgerufen wird.

7.2 Vereinbarung von Funktionen

Die Vereinbarung einer Funktion besteht in C aus zwei Teilen:

- In einer *Deklaration* werden die Eigenschaften einer Funktion bekanntgemacht: Name, Parameterliste und Typ des Funktionswerts.
- In der *Definition* werden die Operationen festgelegt, Speicherplatz reserviert, usw.

Es ist offensichtlich, dass die Definition einer Funktion ihrer Deklaration entsprechen muss.

Die Funktionsdeklaration hat in ANSI-C die Form

```
typ name (typ1 [parameter1], ..., typN [parameterN]);
```

und wird als *Prototyp* bezeichnet.

Für Funktionsnamen gelten dieselben Regeln wie für Namen von Variablen und Konstanten. Wenn eine Funktion keine Parameter besitzt, muss dieses durch das Schlüsselwort `void` gekennzeichnet werden.

Beispiel: Wir wollen eine Funktion schreiben, die die Potenz x^y einer `double`-Zahl x und einer ganzen Zahl y berechnet ($y \geq 0$). Der Prototyp kann dann die Form

```
double potenz (double basis, int exponent);
```

oder

```
double potenz (double, int);
```

haben. Obwohl auch die zweite Deklaration den Intentionen von ANSI-C vollständig entspricht, wie wir noch sehen werden, ist doch die erste Deklaration vorzuziehen, weil sie nicht nur die Typen der Parameter, sondern durch die Wahl der Namen auch deren Bedeutung beschreibt.

In der Definition einer Funktion werden die Operationen festgelegt, die beim Aufruf der Funktion durchzuführen sind. Die Funktionsdefinition hat die Form

```
typ name (typ1 parameter1, ..., typN parameterN) {
    lokale deklarationen
    anweisungsfolge
}
```

Man beachte: In der Definition einer Funktion wird der *Funktionsheader* nicht wie in der Deklaration durch ein Semikolon abgeschlossen!

Als erstes Beispiel realisieren wir die bereits angesprochene Funktion **potenz**:

```
double potenz(double basis, int exponent) {
    double pot = 1;

    for (; exponent > 0; exponent--)
        pot *= basis;

    return pot;
}
```

Als weiteres Beispiel betrachten wir eine Funktion **strlen**, die die Länge eines String bestimmt, ohne das abschließende Null-Zeichen mitzuzählen:

```
int strlen(char s[]) {
    int i;

    for (i = 0; s[i] != '\0'; i++)
        ;

    return i;
}
```

Dabei haben wir benutzt, dass sich die Länge des übergebenen Strings aus seinem Inhalt ergibt: Das Null-Zeichen zeigt das Ende an. Im allgemeinen muss bei Feldern als Funktionsparameter jedoch die Länge des Feldes übermittelt werden, z.B. durch einen separaten Parameter.

Zwei Anmerkungen noch zu grundlegenden Formalien:

- Eine Funktionsdefinition darf nicht innerhalb einer anderen Funktionsdefinition stehen, d.h. C erlaubt keine Schachtelung von Funktionsdefinitionen. Für Funktionsaufrufe gilt das nicht, wie wir bereits gesehen haben: Funktionen können ohne weiteres andere Funktionen aufrufen.
- Deklarationen, die innerhalb eines Funktionsrumpfes vereinbart werden, (z.B. in den beiden Beispielen jeweils die Variable *i*) sind in der Regel lokal, d.h. sie sind nur innerhalb der jeweiligen Funktion bekannt. Dass diese Einschränkung äußerst sinnvoll ist, werden wir später noch im einzelnen sehen.

7.3 Funktionswerte

Welchen *Funktionswert* eine Funktion an die rufende Funktion zurückliefert, wird durch die `return`-Anweisung

```
return [ausdruck];
```

festgelegt. Stimmt der Typ des Ausdrucks *ausdruck* nicht mit dem Typ der Funktion überein, erfolgt eine Typumwandlung wie bei einer Wertzuweisung.

Wie bereits erwähnt, ist die pauschale Bezeichnung „Funktion“ für alle Unterprogramme eigentlich „Etikettenschwindel“ – eine Funktion braucht nämlich durchaus keinen Funktionswert zurückzuliefern! Dieses kennzeichnet man in Deklaration und Definition, indem man als Typ des Funktionswertes das Schlüsselwort `void` angibt. Außerdem entfällt in der `return`-Anweisung der Ausdruck; eine `return`-Anweisung ohne Ausdruck direkt vor der schließenden Klammer des Funktionsrumpfes kann man sogar ganz weglassen, da die Kontrolle bei ihrem Erreichen auf jeden Fall an die aufrufende Funktion zurückgeht.

Ein triviales Beispiel für eine Funktion, die weder einen Parameter besitzt noch einen Funktionswert liefert, ist die „leere“ Funktion

```
void nichts_tun(void) {  
    return;  
}
```

Ein ähnlich triviales Beispiel ist eine Funktion, die ein mehrfaches Piepen bewirkt (falls das Terminal dazu imstande ist). Sie hat als Parameter die Anzahl der Piep's und liefert ebenfalls keinen Funktionswert:

```
void piepen(int anzahl) {  
    while (anzahl--)  
        putchar ('\a');  
}
```

Gibt eine Funktion, deren Funktionswert nicht den Typ `void` besitzt, keinen Funktionswert zurück, weil die `return`-Anweisung oder auch nur der Ausdruck hinter `return` fehlt, so ist das in der Regel ein *schwerer Fehler*. Allerdings: Der Standard schreibt nicht vor, wie ein Compiler hierauf zu reagieren hat.

Faktisch ist die Angabe des Typs einer Funktion in Deklaration und Definition optional: Bei jeder Funktion, bei der die explizite Angabe des Typs fehlt, nimmt der Compiler den Typ `int` an! Der Hintergrund ist, dass man bei Compilern (und nicht nur bei ihnen) nach Möglichkeit versucht, *Aufwärtskompatibilität* zu wahren, d.h. ein Compiler nach neuem Standard soll auch Programme nach altem Standard übersetzen, ohne dass Änderungen erforderlich werden. Leider können durch dieses Prinzip alle Absicherungen, die der ANSI-Standard vorsieht, unterlaufen werden. Programmierer – gerade Programmieranfänger – sind gut beraten, sich strikt an die Vorgaben des ANSI-Standards in strengster Auslegung zu halten. Moderne Compiler unterstützen dies durch entsprechende Optionen. Ein solches Vorgehen sichert Portabilität und hilft enorm bei der Fehlersuche.

7.4 Aufruf von Funktionen

Der *Aufruf* einer Funktion besteht aus dem Namen der Funktion, dem, in runde Klammern eingeschlossen, die Liste der *Funktionsargumente* folgt. Das sind die Werte, die die Funktion während ihrer Ausführung als Parameter haben soll.

Funktionsaufrufe kennen wir bereits aus dem ersten Programmbeispiel. Unsere Funktionsbeispiele aus diesem Abschnitt könnten wir etwa durch

```
i = potenz(2.0, 5);
j = strlen("Wie lang ist dieser String?");
nichts_tun();
```

aufrufen. Durch ihren Aufruf geht die Kontrolle auf die Funktion über, bis diese eine weitere Funktion aufruft oder die Kontrolle an die rufende Funktion zurückgibt.

Zulässig, wenn auch sinnlos, wären ebenso die Aufrufe

```
potenz(2.0, 5);
strlen("Wie lang ist dieser String?");
```

nicht jedoch

```
z = nichts_tun();
```

Liefert eine Funktion einen Funktionswert, so darf man diesen ignorieren; bei einer Funktion, die keinen Funktionswert liefert, kann man auf einen Funktionswert natürlich auch nicht zugreifen.

7.5 Parameter und Argumente

Wir haben bereits gesehen, wie die Kommunikation zwischen rufender und gerufener Funktion erfolgt: In der Definition einer Funktion können *Parameter* (auch: *formale Parameter*) definiert werden. Diese Parameter stehen innerhalb der Funktion wie lokale Variablen des entsprechenden Typs zur Verfügung. Welche Werte sie haben, wird beim Aufruf der Funktion durch die *Argumente* (auch: *Aktualargumente* genannt) festgelegt.

Die Argumente werden den Parametern linear zugeordnet; die einzelnen Zuordnungen erfolgen wie Wertzuweisungen. Damit sollte offensichtlich sein, dass die Anzahl der Argumente mit der Anzahl der Parameter übereinstimmen muss und dass die Typen der Argumente ggf. wie bei einer Wertzuweisung in die Typen der Parameter umgewandelt werden. Die eventuellen Typumwandlungen schreibt der ANSI-Standard vor.

Betrachten wir unter diesem Aspekt noch einmal den Funktionsheader der Funktion `potenz` von oben

```
double potenz(double basis, int exponent);
```

und ihren Aufruf

```
potenz(a, j)
```

Die Funktion `potenz` hat die Parameter `basis` und `exponent`. Beim Aufruf wird `basis` der Wert von `a` und `exponent` der Wert von `j` zugewiesen.

Die Parameter einer Funktion entsprechen lokalen Variablen des jeweiligen Typs, in die beim Aufruf die Werte der Argumente kopiert werden. Man bezeichnet das als „*call by value*“. Die Konsequenzen müssen wir uns noch einmal klar machen: Eine Veränderung eines Parameters innerhalb der Funktion hat *keinerlei* Auswirkungen auf die Umwelt der Funktion! Wir haben dieses im Zusammenhang mit der Funktion `potenz` bereits doppelt genutzt:

- Die Funktion verwendet den Parameter `exponent` als Schleifenzähler und verändert ihn dabei. Das hat, wenn wir die Funktion durch

```
potenz (a, j)
```

aufrufen, aber keinerlei Folgen für die Variable `j` der rufenden Funktion, weil ja nur eine Kopie des Wertes von `j` an `potenz` übergeben wird.

- Der Aufruf

```
i = potenz(2.0, 5);
```

ist nur deshalb zulässig, weil die Funktion nicht mit den Argumenten selbst arbeitet, sondern mit lokalen Variablen, in die die Werte der Argumente kopiert werden. Würde die Funktion mit den Argumenten selbst arbeiten, wäre der Aufruf unzulässig, weil das zweite Argument, das in der Funktion verändert wird, als Konstante keine Adresse besitzt.

Nicht in jedem Falle ist es zweckmäßig und wünschenswert, dass Funktionen ihre Parameter nicht verändern können. Mit der Funktion `scanf` kennen wir auch bereits ein Beispiel für eine solche Funktion: Sie soll ja gerade in den Variablen, die als Argumente angegeben werden, die gelesenen Werte speichern! Entsprechend müssen in solchen Fällen die *Adressen* der Argumente anstelle ihrer *Werte* übergeben werden. Wir haben auch bereits gesehen, wie man genau das mit dem Adressoperator `&` erreichen kann.

Übergabe von Adressen bezeichnet man allgemein als „*call by reference*“. Von C wird behauptet, dass es nur „*call by value*“ kenne, nicht jedoch „*call by reference*“. Der Hintergrund ist ein kleiner sprachlicher „Klimmzug“: Schreiben wir für eine `int`-Variable etwa

```
scanf("%d", &i)
```

so betrachtet C `&i` nicht als „Adresse der Variablen `i`“, sondern als „Ausdruck mit einem Zeigertyp“ – und was dann übergeben wird, ist eben gerade der Wert dieses Ausdrucks.

Auf die Zeigertypen und Zeiger kann an dieser Stelle noch nicht im einzelnen eingegangen werden. Zunächst nur ein Hinweis: Es reicht natürlich *nicht* aus, einen Zeiger an eine Funktion zu übergeben, um ihr die Veränderung ihrer Parameter zu ermöglichen. Auch die Vereinbarung der Parameter und der Zugriff auf sie müssen entsprechend formuliert sein. Wir werden darauf später noch ausführlich zurückkommen.

7.6 Felder als Parameter

Felder nehmen als Parameter (und Argumente) eine Sonderstellung ein. In der Terminologie von C gilt: Der Name eines Feldes ist eine Zeigerkonstante, deren Wert die Adresse der ersten Komponente des Feldes ist.

Die Konsequenz ist, dass bei Feldern nie eine Kopie der Werte der Komponenten an eine Funktion übergeben wird, sondern stets nur die (Anfangs-)Adresse des Feldes. Sieht man also bei einer Funktion ein Feld als Parameter vor, so muss man darauf achten, dass man nicht unerwünschte Nebeneffekte durch die Veränderung von Feldkomponenten erzeugt.

Als Beispiel für eine Funktion mit einem Feld als Parameter haben wir bereits die Funktion `strlen` betrachtet. Sie greift zwar auf die Komponenten ihres Parameters zu, verändert sie aber nicht; Nebeneffekte treten also nicht auf.

Als weiteres Beispiel wollen wir jetzt eine Funktion betrachten, die einen String invertiert. Die Anweisungsfolge, die diese Aufgabe löst, haben wir bereits betrachtet. Jetzt wollen wir sie als Funktion formulieren. Die Funktion `strlen` werden wir verwenden, um die Länge des String zu bestimmen. Außerdem ist ein Rahmenprogramm zum Lesen und Schreiben der Strings angegeben.

```
#include <stdio.h>

#define LAENGE 20

/** Prototypen *****/
```

```

void invert(char str[]);
int strlen(char str[]);

/*= Rahmenprogramm =====*/
int main(void) {
    char string[LAENGE + 1];
    int anzahl;

    printf("String eingeben (max. %d Zeichen):\n", LAENGE);
    scanf("%s", string);
    invert(string);
    printf("Der invertierte String ist \'%s\'\n", string);

    return 0;
}

/*= Funktionen =====*/

/*- Invertieren eines String -----*/
void invert(char str[]) {
    int i, j, n;
    char c;

    n = strlen(str);
    for (i = 0, j = n - 1; i < j; i++, j--)
        c = str[i], str[i] = str[j], str[j] = c;
}

/*- Bestimmung der Laenge eines String -----*/
int strlen(char str[]) {
    int i = 0;

    while (str[i] != '\0')
        i++;

    return i;
}

```

Quelltext 7.1: Invertieren eines Strings

Einige Anmerkungen dazu:

- Das Rahmenprogramm „verlässt“ sich darauf, dass der Benutzer keinen zu langen String eingibt; dieser String darf keine „white spaces“ enthalten.
- Das Hauptprogramm ist wie üblich als erstes angegeben und erst danach die Funktionen.
- Die Funktionsdeklarationen von `invert` und `strlen` stehen *vor* dem Hauptprogramm. Auch das ist üblich. Wir hätten auch `invert` innerhalb des Hauptprogramms und `strlen` innerhalb von `invert` deklarieren können; das hätte zur Folge gehabt, dass `invert` nur innerhalb des Hauptprogramms und `strlen` nur innerhalb von `invert` zur Verfügung steht.

Übrigens: Nur Feldnamen werden als Zeigerkonstanten betrachtet; für Feldkomponenten gilt das nicht. Rufen wir die Funktion `potenz` etwa durch

```
a = potenz(t[k], 4);
```

auf, so wird der Wert von `t[k]` übergeben und nicht die Adresse.

7.7 Das Attribut `const`

Wir haben eben gesehen, dass eine Funktion bei einem Feld, das als Argument übergeben wird, stets den direkten Zugriff auf die Originaldaten hat und diese ggf. verändern kann.

Das ist ziemlich unschön, weil der Programmierer sehr diszipliniert arbeiten muss, damit nicht ungewollte und ungewünschte Veränderungen erfolgen. War Disziplin früher das einzige Mittel, Nebeneffekte zu verhindern, so gibt der Standard dem Programmierer Unterstützung: Versieht man einen Parameter mit dem Attribut `const`, so erzeugt der Compiler bei jeder Zuweisung an den Parameter, bei Feldern auch an eine Komponente, eine Fehlermeldung. Wohlgemerkt: Der Compiler meldet den Fehler nur; er sorgt nicht automatisch dafür, dass eine Kopie angelegt wird.

Bei Parametern ist das Attribut `const` zunächst nur für Felder sinnvoll, weil man bei „einfachen“ Variablen ohnehin nur eine Kopie des Wertes erhält, die man unbesorgt verändern kann. Die Funktion `strlen` sollte man zum Beispiel so deklarieren:

```
int strlen(const char str[]);
```

Hier mag das überflüssig erscheinen, weil man auf den ersten Blick sieht, dass im Rumpf der Funktion nichts „Böses“ passiert. Überlegen Sie sich aber auch dieses:

- Bei vielen Funktionen, etwa den Funktionen der Standardbibliothek, kann man sich nur die Deklarationen in einer Headerdatei und ggf. eine zugehörige Beschreibung ansehen, während die Definition nicht zugänglich ist.
- Je länger eine Funktion ist, desto mühsamer wird die Prüfung, ob sie einen Parameter verändert oder nicht.

In beiden Fällen ist es angenehm, schon aus dem Prototyp ablesen zu können, ob die Funktion ein übergebenes Feld unverändert lässt.

Das Attribut `const` darf man übrigens auch für Variablen vergeben. Dann kann es auch für einfache Variablen sinnvoll sein.

7.8 Prototypen

Prototypen, d.h. Funktionsdeklarationen mit Angabe der Parameterliste, haben verschiedene Zwecke:

- Der Compiler kann prüfen, ob Argumente und Parameter einer Funktion übereinstimmen oder doch zumindest zueinander „passen“, ohne den für die Funktion auszuführenden Code zu kennen. Das ist häufig dann der Fall, wenn die Funktion aus einer Bibliothek stammt, deren Quelltext nicht veröffentlicht wurde.
- Der Compiler kann, soweit erforderlich und möglich, die Typen der Argumente in die Typen der Parameter umwandeln. Die Umwandlung erfolgt, wie bereits angesprochen, nach denselben Regeln wie bei Wertzuweisungen.
- Der Compiler kann einen Funktionswert bei Bedarf in einen anderen Typ umwandeln.

Es sollte klar sein: Damit die Prüfungen und ggf. Umwandlungen ausgeführt werden können, muss der Prototyp einer Funktion in der Quelldatei vor ihrem ersten Aufruf stehen. Die Definition kann dann an beliebiger anderer Stelle folgen.

7.9 Rekursion

C erlaubt *Rekursion*. Man spricht von

- *direkter Rekursion*, wenn eine Funktion sich selbst aufruft, und von
- *indirekter Rekursion*, wenn sich zwei oder mehr Funktionen wechselseitig aufrufen.

Rekursion wird in der Mathematik häufig in Definitionen verwendet. Das „klassische“ Beispiel ist die Berechnung der Fakultät $n!$ einer nichtnegativen ganzen Zahl n :

$$n! := \begin{cases} n \cdot (n-1)! & \text{für } n > 0 \\ 1 & \text{für } n = 0 \end{cases}$$

Mit rekursiven Funktionen lassen sich solche Formeln direkt umsetzen:

```
long int fakultaet(int n) {
    if (n < 0)          /* Fehler */
        return 0;

    if (n == 0)         /* Abbruchbedingung */
        return 1;

    return n * fakultaet(n - 1);
}
```

Der Programmierer muss dafür sorgen, dass die Rekursion (irgendwann) „abbricht“. Im Beispiel sorgen die Abbruchbedingung $n > 0$ und der rekursive Aufruf mit dem um eins verringerten Argument gemeinsam dafür.

Ob man rekursiv formulierte Berechnungen auch rekursiv realisieren sollte, steht auf einem anderen Blatt. Rekursion ist in der Regel ziemlich aufwendig: Die Parameter müssen kopiert werden („call by value“), lokale Variablen müssen bereitgestellt, der Unterprogramm-sprung muss ausgeführt werden. Man sollte sich deshalb zunächst einmal überlegen, ob sich eine rekursiv formulierte Aufgabe nicht auch ohne weiteres *iterativ*, d.h. mit einer Schleife, lösen lässt.

Auch unter diesem Aspekt ist die Berechnung der Fakultät einer nichtnegativen ganzen Zahl das „klassische“ Beispiel. Man kann sie nämlich durch

$$n! = \prod_{i=1}^n i$$

definieren und dann wie folgt realisieren.

```
long int fakultaet (int n) {
    long int fak = 1;

    if (n < 0)          /* Fehler */
        return 0;

    while (n > 1)
        fak *= n--;

    return fak;
}
```

Ob eine rekursive oder eine nicht-rekursive Realisierung bestimmter Funktionen vorzuziehen ist, hängt sehr vom Einzelfall ab. Immer dann, wenn sich Rekursion, wie bei den Fakultäten, durch eine einfache Schleife oder ein ähnlich einfaches anderes Konstrukt vermeiden lässt, sollte man sie auch vermeiden. Andererseits gibt es Fälle, in denen Rekursion

nur mit erheblichen „Klimmzügen“ zu vermeiden wäre – und dann sollte man sie auch nutzen.

Die folgenden Beispiele sollten ausreichen, um ein grundlegendes Verständnis für rekursive Algorithmen zu bekommen. Eine ausführlichere Behandlung rekursiver Algorithmen und Techniken kann hier nicht erfolgen.

7.10 Beispiel: Türme von Hanoi

Nicht in jedem Falle gibt es zu einem rekursiv formulierten Problem so offensichtlich eine iterative Lösung wie bei den Fakultäten.

Ein solches Beispiel sind die „Türme von Hanoi“ (Abbildung 7.1): Man hat einen Turm von Scheiben, bei dem jede Scheibe kleiner ist als die, auf der sie liegt. Der Turm soll versetzt werden, wobei zusätzliche Regeln einzuhalten sind:

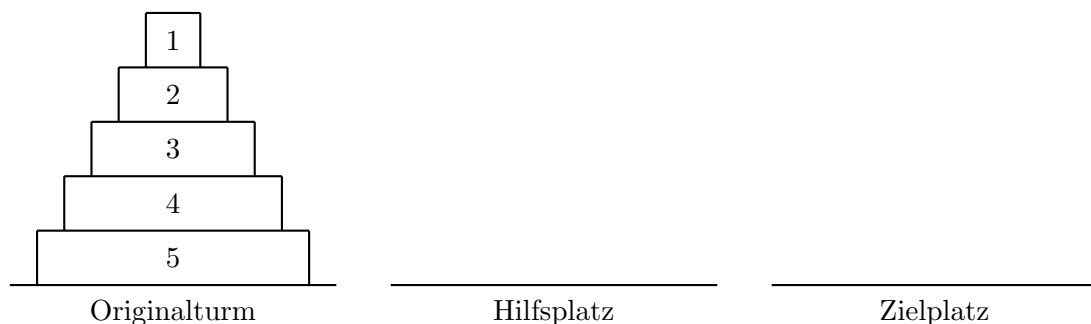


Abbildung 7.1: Türme von Hanoi

- Es darf immer nur die oberste Scheibe eines Turmes bewegt werden.
- Neben dem Ausgangsturm und dem Zielplatz steht ein dritter Platz zur Ablage von Scheiben zur Verfügung.
- Jede Scheibe darf nur auf den Boden oder auf eine größere Scheibe gelegt werden.

Nehmen wir einmal an, dass der Turm aus n Scheiben besteht. Die drei Plätze wollen wir mit „O“ („Originalplatz“), „H“ („Hilfsplatz“) und „Z“ („Zielplatz“) bezeichnen. Dann ist die rekursive Lösung der Aufgabe trivial:

1. Man legt die obersten $n - 1$ Scheiben von „O“ nach „H“.
2. Die unterste Scheibe wird nach „Z“ gelegt.
3. Man legt die $n - 1$ Scheiben von „H“ nach „Z“.

Damit hat man das Problem der n Scheiben auf zwei Probleme mit jeweils $n - 1$ Scheiben zurückgeführt. Diese löst man nach demselben Verfahren, wobei nur die drei Plätze ihre Bedeutung wechseln:

- Im ersten Schritt ist „H“ der „Zielplatz“ für den Turm aus $n - 1$ Scheiben und „Z“ der „Hilfsplatz“.
- Im dritten Schritt ist „H“ der „Originalplatz“ des Turms aus $n - 1$ Scheiben und „O“ der „Hilfsplatz“.

Dass die Rekursion zum Erfolg führt, ist klar: Bei jedem Rekursionsschritt verringert sich die Höhe des zu verschiebenden Turmes um 1 – und für einen Turm der Höhe 1 besteht seine Verschiebung einfach in der Verschiebung seiner einen Scheibe.

Ein Programm, das diesen rekursiven Algorithmus rekursiv realisiert, kann so aussehen:

```

#include <stdio.h>

/** Prototyp *****/
void hanoi(int hoehe, char von, char nach, char ueber);

/* Rahmenprogramm *****/
int main(void) {
    int n;

    printf("Wie hoch ist der Turm? ");
    scanf("%d", &n);
    hanoi(n, 'O', 'Z', 'H');

    return 0;
}

/* rekursives Ausfuehren der Zuege *****/
void hanoi(int hoehe, char von, char nach, char ueber) {
    if (hoehe > 1)
        hanoi(hoehe - 1, von, ueber, nach);

    printf("Bewege Scheibe '%d' nach '%c'!\n", hoehe, nach);

    if (hoehe > 1)
        hanoi(hoehe - 1, ueber, nach, von);
}

```

Quelltext 7.2: Türme von Hanoi rekursiv

An dieser Stelle sei angemerkt, dass es auch für die Lösung dieses Problems einen iterativen Algorithmus gibt.

7.11 Beispiel: Quicksort

Die beiden Beispiele für Rekursion, die wir bislang betrachtet haben, ließen sich auch iterativ lösen. Bei den Fakultäten war das sehr einfach, bei den Türmen von Hanoi erforderte es schon längere Überlegungen.

Es gibt jedoch auch Aufgaben, die sich prinzipiell *nur* rekursiv lösen lassen. Hierzu zählt „Quicksort“, ein Verfahren zum Sortieren von Vektoren.

Quicksort funktioniert so: Wir wählen zunächst ein beliebiges Element des Vektors, z.B. das an seinem Anfang stehende. Jetzt vertauschen wir Elemente des Vektors so, dass

- das ausgewählte Element an seinen endgültigen Platz kommt, und dass
- links von ihm nur nicht größere und rechts von ihm nur nicht kleinere stehen.

Damit sind wir in einer ähnlichen Situation wie bei den „Türmen von Hanoi“: Anstelle eines Vektors mit z.B. n Komponenten haben wir jetzt zwar *zwei* Vektoren zu sortieren – die Gesamtzahl der Komponenten dieser beiden Vektoren ist aber nur $n - 1$. Damit ist jeder der beiden Teilvektoren (echt) kürzer als der Ausgangsvektor und es ist klar, dass der Algorithmus terminiert: Irgendwann im Zuge der Rekursion wird *jeder* Teilvektor auf die Länge 0 oder 1 reduziert – und ein leerer Vektor ist ebenso sortiert wie ein Vektor mit nur einer Komponente.

Damit erweist sich die rekursive Routine selbst wieder als sehr simpel:

```

void quick(int z[], int von, int bis) {
    int p;
    if (von < bis) {
        p = platz(z, von, bis);
        quick(z, von, p - 1);
        quick(z, p + 1, bis);
    }
}

```

Nicht so ohne weiteres hinschreiben kann man die Funktion `platz`. Wir müssen uns zunächst überlegen, wie sie arbeiten soll, und tun das anhand eines Beispiels. In der Zahlenfolge

2 8 7 5 3 6 4 1

soll das erste Element (2) an seinen endgültigen Platz gebracht werden; dabei sollen die übrigen Zahlen so vertauscht werden, dass links von der 2 nur nicht größere Zahlen und rechts davon nur nicht kleinere Zahlen stehen. Dazu speichern wir die 2 zunächst in einer Hilfsvariablen:

2
- 8 7 5 3 6 4 1

Dadurch wird die erste Komponente des Vektors frei. Jetzt suchen wir von ganz rechts die erste Zahl, die kleiner als 2 ist, und speichern sie auf den freien Platz um:

2
1 8 7 5 3 6 4 -

Dadurch wird deren alter Platz frei. Jetzt suchen wir, rechts neben der umgespeicherten Zahl beginnend, aufwärts nach der ersten Zahl, die größer als 2 ist, und speichern sie auf den freien Platz um:

2
1 - 7 5 3 6 4 8

Dadurch wird deren alter Platz frei. Jetzt suchen wir wieder, links neben der umgespeicherten Zahl beginnend, abwärts nach der ersten Zahl, die kleiner als 2 ist, und speichern sie auf den freien Platz um. Und diese abwechselnde Suche von rechts und links wiederholen wir, bis sich die Indizes überschneiden. Dann haben wir den gesuchten Platz gefunden: Wir können die herausgenommene Zahl an diesem Platz wieder in den Vektor eintragen und den Index des Platzes als Funktionswert zurückliefern.

Die Realisierung kann so aussehen:

```

int platz (int z[], int von, int bis) {
    int h;

    h = z[von];
    while (von < bis) {
        while (von < bis && z[bis] >= h)
            bis--;

        if (von < bis) {
            z[von] = z[bis];
            von++;
        }
    }
}

```

```

    while (von < bis && z[von] <= h)
        von++;

    if (von < bis) {
        z[bis] = z[von];
        bis--;
    }
}
z[von] = h;
return von;
}

```

Was den Quicksort von den Fakultäten und den „Türmen von Hanoi“ unterscheidet, ist die Tatsache, dass Daten gespeichert werden müssen: Haben wir den Ausgangsvektor unterteilt, können wir nur einen der beiden Teilvektoren sofort weiter verarbeiten; Anfangs- und Endindex des anderen Teilvektors müssen wir speichern. Für den Teilvektor, den wir sofort weiterverarbeiten, gilt dasselbe: Erneut können wir nur einen seiner Teile sofort weiterverarbeiten; Anfangs- und Endindex des anderen Teils müssen wir speichern.

Formulieren wir den Algorithmus rekursiv, so sorgt der Compiler für die Speicherung der Anfangs- und Endindizes der noch zu sortierenden Teilvektoren. Versuchen wir, den Algorithmus iterativ zu lösen, müssen wir selbst für die Speicherung der Anfangs- und Endindizes der noch zu sortierenden Teilvektoren sorgen. Eine scheinbar iterative Realisierung des Quicksort ist letztlich also nur Simulation von Rekursion – und dafür spricht wenig.

Quicksort ist für Vektoren ein sehr schnelles Sortierverfahren – sofern die Vektoren nicht schon weitgehend vorsortiert sind. In der hier vorgestellten Variante wird Quicksort für (auf- oder absteigend) sortierte Vektoren allerdings zum extremen „Slowsort“, weil in jedem Schritt die Ausgangsfolge der Länge m in eine leere Teilfolge und eine Teilfolge mit $m-1$ Komponenten zerlegt wird. Am schnellsten ist Quicksort dann, wenn in jedem Schritt die Ausgangsfolge in zwei gleich lange Teilfolgen zerlegt wird. Dem kann man Rechnung tragen, indem man nicht jeweils das erste Element der Teilfolge an seinen endgültigen Platz bringt. Denkbar wäre, stets das mittlere Element zu nehmen, oder auch von erstem, mittlerem und letztem Element das mit dem in der Mitte liegenden Wert.

Auch bei der rekursiven Funktion kann man noch mancherlei „optimieren“:

- Von den beiden rekursiven Aufrufen ist der zweite überflüssig und kann ohne weiteres durch eine Schleife ersetzt werden:

```

void quick(int z[], int von, int bis) {
    int p;
    while (von < bis) {
        p = platz(z, von, bis);
        quick(z, von, p - 1);
        von = p + 1;
    }
}

```

Der erste rekursive Aufruf lässt sich dagegen nicht ohne weiteres ersetzen, weil die Rekursion gerade dafür sorgt, dass die Werte von `p` und `bis` (automatisch) gesichert werden.

- Die maximale Rekursionstiefe kann im „worst case“ (in jedem Schritt Abspalten nur eines Elements) bei n zu sortierenden Zahlen $n-1$ erreichen. Das kann man verhindern, indem man jeweils zunächst die kürzere Teilfolge durch Rekursion, danach die

längere Teilfolge durch Iteration sortiert:

```
void quick(int z[], int von, int bis) {
    int p;
    while (von < bis) {
        p = platz(z, von, bis);
        if (p - von < bis - p) {
            quick(z, von, p - 1);
            von = p + 1;
        }
        else {
            quick(z, p + 1, bis);
            bis = p - 1;
        }
    }
}
```

Jetzt kann die Rekursionstiefe bei zunächst n Zahlen nur noch maximal $\log_2 n$ erreichen.

- Die Funktion lässt sich auch mit nur zwei Parametern formulieren, wenn man nicht mit den Anfangs- und Endindizes, sondern mit Zeigern auf die entsprechenden Komponenten arbeitet. Hierfür fehlen uns derzeit aber noch die Mittel.

Wir haben jetzt drei Beispiele für Rekursion betrachtet:

- Bei den Fakultäten haben wir gesehen, dass Rekursion wenig sinnvoll ist.
- Bei den „Türmen von Hanoi“ gibt es zwar eine nichtrekursive Lösung. Ob diese aber effizienter als die rekursive Lösung ist, müsste man erst genauer untersuchen.
- Beim Quicksort ist jede scheinbar iterative Lösung tatsächlich nur Simulation von Rekursion.

7.12 Beispiel: Potenzen

Die bisherige Implementierung der Potenzfunktion lässt sich noch weiter optimieren. Dazu verwendet man die Binärdarstellung des Exponenten.

$$x^{109_d} = x^{1101101_b} = x^{1 \cdot 64} \cdot x^{1 \cdot 32} \cdot \underbrace{x^{0 \cdot 16}}_{=1} \cdot x^{1 \cdot 8} \cdot x^{1 \cdot 4} \cdot \underbrace{x^{0 \cdot 2}}_{=1} \cdot x^{1 \cdot 1}$$

Man berechnet also iterativ die Potenzen $x^{(2^k)}$, $k = 0, 1, \dots$ und entscheidet in jedem Schritt, ob der Faktor am Ergebnis beteiligt ist. Das erreicht man mit dem Modulo-Operator. Anschließend wird der Exponent durch zwei geteilt, um im nächsten Schritt das nächste Bit ermitteln zu können – Division mit zwei entspricht dem Schieben der Bits um eine Stelle nach rechts.

```
double potenz(double basis, unsigned exponent) {
    double p = 1;

    while (exponent != 0) {
        if (exponent % 2)
            p *= basis;
        exponent /= 2;
        basis *= basis;
    }
    return p;
}
```

Diese iterative Lösung kommt mit maximal $\log_2(\text{exponent})$ Schritten aus. Das ist ein enormer Gewinn gegenüber linearem Aufwand.